AFP-Enterprise: Host Runtime Kernel for Planner–Tool Stacks

Muchen He
Independent Researcher

7 October 2026

Abstract

AFP-Enterprise is a host runtime kernel beside a local planner, reached by a synchronous pre-flight probe. Before irreversible tool I/O, the planner reports recursion depth and context, then blocks on allow, delay, or isolate. The kernel does not parse prompts or tool schemas. It checks, in order, an emergency stop, a depth cap, a four-axis pressure maximum, and a specified isolate–recover state machine.

On a personal computer, a LangGraph planner issued real lookup and loopback HTTP GET calls. Five bounded jobs completed under no gate, a per-step wipe, and the current sidecar (false isolate = 0). A scripted loop and a 48-task burst ran to completion without a gate and were contained by both gated variants. Those traces fire the depth cap and the hard pressure isolate; they do not reach the delay band or the recover machine. The gated variants differ only when a later probe reuses leftover host telemetry: AFP-Enterprise refuses; the wipe allows. A new job that reports depth 0 is admitted under both. Two planners on one sidecar can overwrite each other, so the kernel keeps last-reported host telemetry, not a lock on who the planner is.

The measured increment is retained execution telemetry, not identity-locked persistence, production safety, or LLM-agent validation. A 500-node Monte Carlo in the appendix is a breaker invariant, not a field result.

Keywords: multi-agent systems; planner–tool runtime; admission control; pre-flight check; retained telemetry; stale-state retention; circuit breaker.

1 The empty cell

Multi-agent infrastructure has matured along two axes: planning (LangGraph, CrewAI, AutoGen) and transport (gRPC, HTTP, Kafka, MCP). A third cell is empty. Before each hop commits, something must answer: may this step execute?

MCP and A2A decide *what* may be said. They do not decide whether the next hop may run. Per-request gateways forget. Token budgets often account after commit. Framework retry and recursion limits reset when the session ends. In-process guardrails share the optimizer's address space.

The failure geometry is stacking: planner $\to$ tool $\to$ planner $\to \cdots$, sometimes closing a loop. Each hop can be a valid tool call while depth, context, concurrency, and burst grow until the node saturates.

Three default pathologies:

- **Intent burst.** One objective fans into tool chains. Wire metrics can stay quiet.
- **Recursive delegation loop.** Replan / reflect / retry stays syntactically valid while depth climbs.
- **Context avalanche.** Bounded depth does not bound state volume.

AFP-Enterprise adjudicates the physical question—will this step blow through depth, pressure, or burst?—not hallucination and not task correctness. The scene is a local planner plus MCP or local tools on hosts the operator already controls.

2 What AFP-Enterprise is

Legal planner steps can still stack a machine. AFP-Enterprise is a host kernel that answers, before the tool runs, whether this step may proceed. Pre-flight is how the planner calls that kernel.

The planner does not embed the kernel. Each hop is write-then-probe:

1. report recursion depth and context bytes;
2. probe with a burst hint (estimated imminent fan-out);
3. proceed only on allow; sleep on delay; halt I/O on isolate.

The probe must finish before irreversible I/O. There is no advisory-only mode. The reference placement is an out-of-process sidecar (Unix-domain socket and gRPC).

It is not alignment or injection defense, not a grader of answers, not a network ingress filter, and not validated field safety. The delay band and isolate–recover machine are specified; these traces do not reach them. The shipped sidecar keeps last-reported depth, not a lock on agent identity.

3 Mechanism

3.1 Timing

An action is pre-intent if and only if it is evaluated before the planner commits a tool call, delegation, or outbound message, or before irreversible I/O. Parsing RPC after the fact is observation, not this kernel. The contested resource is optimization capacity, not prompt text.

Each hop:

1. `ReportInternalState(recursion_depth, context_bytes)`
2. `PreFlightCheck(estimated_tasks, trace_id)`
3. allow → execute; isolate → stop. Do not convert isolate into an ungoverned retry storm.

If the state report is omitted or stale, pressure at probe time is under-measured. An optional destination field on the probe is ignored. State is kept on the host sidecar, not keyed to one agent identity.

3.2 Four stages

The kernel returns on the first match.

1. *Emergency stop.* An operator kill switch isolates every hop.
2. *Depth cap.* Reported depth > `MaxRecursionDepth` (default 10) isolates immediately.
3. *Pressure isolate.* If $e \geq e^\star$ (default 0.95), isolate before the recover machine.
4. *Isolate–recover machine.* Only if the first three stages pass.

These traces fire stage 2 (the loop) or stage 3 (the burst).

3.3 System pressure

Pressure $e \in [0, 1]$ is a maximum, not an average. If any axis saturates, pressure saturates.

$$e \;=\; \max\big(e_{\text{tool}},\, e_{\text{mem}},\, e_{\text{ctx}},\, e_{\text{burst}}\big). \tag{1}$$

Axis	Formula	Default	Captures
Tool	$e_{\text{tool}} = \min(1,\, n_{\text{active}}/C)$	$C = 50$	in-flight tool calls
Memory	$e_{\text{mem}} = \min(1,\, \text{mem}/r_{\text{oom}})$	$r_{\text{oom}} = 0.90$	host memory / OOM
Context	$e_{\text{ctx}} = \min(1,\, b/B)$	$B = 512\,\text{MiB}$	reported planning bytes
Burst	$e_{\text{burst}} = \min(1,\, \hat{n}/C)$	$C = 50$	prospective fan-out

n_{active} is the live tool-call counter. mem is the host memory-usage ratio. b is last-reported context bytes. $\hat{n}$ is `estimated_tasks`; if the hint is 0, the monitor substitutes 1. The storm job $\hat{n} = 48$ gives $e_{\text{burst}} = 0.96$, so S4 isolates on stage 3 with zero GETs.

Band	Value	Effect
Safe e_{safe}	0.40	machine may return to allow
Warn e_{warn}	0.75	delay path (stage 4)
Breaker $e^\star$	0.95	hard isolate (stage 3)

Depth is a discrete breaker, not folded into e:

$$\texttt{reported_depth} > 10 \;\Rightarrow\; \text{ISOLATED}.$$

S3 reports depth 11 and isolates here.

3.4 Delay injection (specified; not reached)

If stage 4 chooses delay,

$$\delta(e) \;=\; \begin{cases} 500\,\text{ms} & e \leq e_{\text{warn}}, \\ 500 + 1500 \cdot \dfrac{e - e_{\text{warn}}}{1 - e_{\text{warn}}}\,\text{ms} & e > e_{\text{warn}}. \end{cases} \tag{2}$$

Delay is 500 ms at the warn line and 2000 ms as $e \to 1$. A probation probe uses a fixed 1000 ms window.

3.5 Isolate–recover machine (specified; not reached)

Four states: allow, delay, isolate, probation. Floor rules fire first:

$$\text{invalid sign} \ \vee \ \text{malicious spike} \ \vee \ v < 0.3 \ \Rightarrow \ \text{isolate}.$$

A malicious spike is $\hat{n} > C$. S4 uses $\hat{n} = 48 < 50$, so the spike rule does not fire; the pressure breaker does.

Current	Condition	Next	Action
Allow	$e > 0.75$	Delay	sleep $\delta(e)$
Allow	otherwise	Allow	proceed
Delay	$e < 0.40$	Allow	proceed
Delay	otherwise	Delay	sleep $\delta(e)$
Isolate	epochs since entry ≥ 64	Probation	1000 ms probe
Isolate	otherwise	Isolate	halt
Probation	$e > 0.40$	Isolate	halt; clock restamps
Probation	dwell > 128 and $v \geq 0.8$	Allow	proceed
Probation	otherwise	Probation	low-rate probe

Leaving delay requires $e < 0.40$, not merely $e < 0.75$. The isolate clock stamps on entry; later dropped probes must not refresh it. On the current local sidecar the recover score is held at 1.0 for the single host bucket. Formula A is idle on these traces.

3.6 Recover-score formulas (specified; idle)

A — evolution ($\alpha = 0.95$, $\beta = 0.05$):

$$v_{\text{new}} \ = \ \text{clamp}\big(\alpha v_{\text{old}} + \beta s - \gamma p - \delta e,\ 0,\ 1\big), \tag{3}$$

with shipped coefficients $\gamma = e^{p} - 1$ and $\delta = 0.1\, e$. s is throughput success; p is a destabilization penalty.

B — idle decay ($\lambda = 0.01$ per epoch):

$$v_{\text{eff}} \ = \ v_{\text{hist}} \cdot e^{-\lambda \Delta t}. \tag{4}$$

C — logarithmic recovery (floor 0.3, $\kappa = 0.02$):

$$v_{\text{rec}}(\Delta t) \ = \ \min\big(1,\ 0.3 + \kappa \log(1 + \Delta t_{\text{probation}})\big). \tag{5}$$

$v < 0.3$ isolates. Full return also requires $v \geq 0.8$ after 128 probation epochs. These traces do not validate A–C or the dwells.

4 Shipped retention

Older drafts stored isolate on identity until asymmetric recovery. The shipped sidecar is not that.

Depth isolation follows last-reported recursion depth. A new graph that reports depth 0 is admitted after isolate. A per-step wipe that clears depth at request start matches the current

sidecar on the bounded and first-containment jobs. They differ only on a stale post-isolate probe that does not rewrite depth.

Claim	Holds now?
Pre-flight stops the tested loop and burst	Yes, on those jobs
Tested bounded jobs are not false-isolated	Yes, on those jobs
Consequences stay on identity until recovery dwell	No
Two planners on one sidecar do not overwrite each other	No

The accurate object is stale-state retention.

5 Empirical record

All runs used one personal computer, one LangGraph, and two real tools: a local key lookup and HTTP GET against a loopback fixture. The primary planner is deterministic so sidecar admission is isolated from stochastic LLM routing; the live 0.5B arm is Appendix B. Sidecar policy: recursion cap 10, concurrency cap 50. No unfinished row is dropped. Latency is indicative only.

5.1 Primary jobs

Table 1 is no-gate versus current sidecar. S1 and S2 finish with the gate on (false isolate = 0). S3 without the gate walks to the graph cap (depth 40, 39 ticks); with it, isolation fires at reported depth 11. S4 without the gate performs the expand plus 48 GETs; with it, $e_{\text{burst}} = 0.96$ opens the breaker and no GET runs.

Bounded completion extends to five jobs: S1, S2, lookups only, fetches only, and eight repeats of one lookup. All five complete under no gate, the wipe, and the current sidecar. Mean wall-clock on that set: 3.8 ms off, 6.4 ms wipe, 6.0 ms current sidecar.

Table 1: No-gate versus current sidecar. A per-step wipe matches the sidecar on these four jobs.

Job	Gate	Done	Tools	Probes	Isolated	Depth	Reason
S1 short	off	yes	1	0	no	2	—
S1 short	on	yes	1	3	no	2	—
S2 long	off	yes	8	0	no	9	—
S2 long	on	yes	8	17	no	9	—
S3 runaway	off	to cap	39	0	no	40	—
S3 runaway	on	no	10	22	yes	11	recursion cap
S4 storm	off	yes	49	0	no	50	—
S4 storm	on	no	0	2	yes	1	burst entropy

5.2 Three-way baseline

Table 2 is the comparison a gate-versus-off table conceals. S1–S4 do not distinguish the two gated variants. They differ only when a later execution reuses leftover telemetry. Appendix D lists every job row.

After S3 isolate, a probe that does not rewrite depth is still refused; a new graph that reports depth 0 is admitted (Appendix A). Request completion does not clear leftover depth.

Table 2: Three-way baseline. Gated columns differ only on a stale post-isolate probe.

Scene	No gate	Per-step wipe	Current sidecar
Five bounded jobs	done	done	done
False isolate	0	0	0
Loop / 48-burst	not stopped	stopped	stopped
Stale probe, no rewrite	—	allow	**refuse**
New job reports depth 0	—	allow	allow

Two planners on separate sidecars keep the second planner healthy while the first isolates on S3. On one shared sidecar, the second planner's fresh report overwrites the first planner's leftover depth (Appendix A). Isolation is a host-level check. If two planners must not affect each other, they need separate sidecars, or the kernel must account per identity first.

6 Related work

Admission control and circuit breaking are standard operations practice [1, 2, 3, 4, 5]. AFP-Enterprise does not invent that primitive. It attaches a host kernel—depth cap, four-axis pressure, retained telemetry, and a specified recover path—before a planner–tool hop, and says which stages the traces reach.

Agent frameworks optimize task completion [6, 7, 8]. Safety evaluations grade harmfulness, injection, and tool abuse [11, 12, 13]. Control and oversight constrain intent [14]. MCP and A2A secure tools and sessions [9, 10]. A valid tool call can still be a physically illegal next hop.

7 Limitations

The evidence is a controlled planner–tool environment on one machine. It is not computer-use, MCP ingress, injection robustness, multi-agent isolation, or production safety. The primary planner is scripted; the live 0.5B arm does not finish the long recipe. Jobs are deterministic, so the tables have no confidence intervals. The emergency stop, delay band, and isolate–recover machine were not reached. State ownership fails on a shared sidecar. These traces stop a false persistence claim. They are not a complete runtime-safety benchmark.

8 Conclusion

AFP-Enterprise is a host runtime kernel for a local planner, reached by a pre-flight probe. On the jobs that were tested, bounded work can finish, and a loop or burst can be stopped by the depth cap or the hard pressure isolate. A per-step wipe can do the same on those jobs. What the current sidecar adds is refusal when leftover host telemetry is not overwritten. The delay band and the isolate–recover machine are specified; they are not evidenced by these tables.

References

[1] V. Jacobson. Congestion avoidance and control. *SIGCOMM*, 1988.

[2] M. Welsh, D. Culler, and E. Brewer. SEDA. *SOSP*, 2001.

[3] M. Nygard. *Release It!* Pragmatic Bookshelf, 2018.

[4] M. Fowler. CircuitBreaker. 2014.

[5] Envoy Proxy. Circuit breaking. 2024.

[6] S. Yao et al. ReAct. *ICLR*, 2023.

[7] Q. Wu et al. AutoGen. 2023.

[8] LangChain. LangGraph. 2025.

[9] Anthropic. Model Context Protocol. 2024.

[10] Google. Agent2Agent protocol. 2025.

[11] K. Greshake et al. Not what you've signed up for. 2023.

[12] E. Debenedetti et al. AgentDojo. 2024.

[13] M. Andriushchenko et al. AgentHarm. 2024.

[14] R. Greenblatt et al. AI control. 2024.

[15] M. Cemri et al. Why do multi-agent LLM systems fail? 2025.

A Stale telemetry and two planners

Table 3: Same sidecar after S3 isolate.

Arm	Isolated	S1 done	Leftover depth	Reason
Stale probe (no rewrite)	yes	—	11	recursion cap
Fresh S1 (reports depth 0)	no	yes	0	—

Table 4: Two local planners. A's S3 is 10 tools / 22 probes; B's S1 is 1 tool / 3 probes in both layouts.

Layout	Agent	Job	Isolated	Done
Split sidecars	A	S3	yes	no
Split sidecars	B	S1	no	yes
Shared sidecar	A	S3	yes	no
Shared sidecar	B	S1	no	yes

B Live run on a local small model

Qwen2.5 0.5B, same graph and tools. S1 is the only completed live job (604 prompt tokens in both modes). S2 without the gate hits LangGraph `recursion_limit`= 30 with 0 tools and 18,859 prompt tokens. S2 with the gate isolates a looping planner at depth 11 (10 tools, 10,799 prompt tokens). Neither cell is a false isolate of a finished eight-tool job.

Table 5: Live AFP-Enterprise on Qwen2.5 0.5B. Supplementary only.

Job	Gate	Done	Tools	Isolated	Tokens	Reason
S1 short	off	yes	1	no	604	—
S1 short	on	yes	1	no	604	—
S2 long	off	no	0	no	18,859	graph recursion limit
S2 long	on	no	10	yes	10,799	recursion cap

C Depth-only unit check

Before tools were wired, the same sidecar isolated a depth-only planner at the recursion cap.

Table 6: Depth-only LangGraph (no tool I/O). Unit check, not the main evidence.

Job	Gate	Finished	Steps	Probes	Isolated	Depth	Reason
Benign	off	yes	3	0	no	3	—
Benign	on	yes	3	3	no	3	—
Runaway	off	to cap	40	0	no	40	—
Runaway	on	no	11	12	yes	11	recursion cap

D Three-way baseline, full latest run

Table 7: Latest three-way run. Wipe matches the current sidecar on every job row; they differ only on the stale probe.

Job	Gate	Done	Tools	Probes	Isolated	Wall (ms)	Reason
S1 short	off	yes	1	0	no	1.6	—
S1 short	wipe	yes	1	3	no	2.0	—
S1 short	sidecar	yes	1	3	no	2.0	—
S2 long	off	yes	8	0	no	8.3	—
S2 long	wipe	yes	8	17	no	9.4	—
S2 long	sidecar	yes	8	17	no	8.7	—
Lookups only	off	yes	4	0	no	1.5	—
Lookups only	wipe	yes	4	9	no	3.8	—
Lookups only	sidecar	yes	4	9	no	4.2	—
Fetches only	off	yes	6	0	no	5.3	—
Fetches only	wipe	yes	6	13	no	10.3	—
Fetches only	sidecar	yes	6	13	no	8.4	—
Eight repeats	off	yes	8	0	no	2.1	—
Eight repeats	wipe	yes	8	17	no	6.3	—
Eight repeats	sidecar	yes	8	17	no	6.7	—
S3 runaway	off	yes	39	0	no	8.3	—
S3 runaway	wipe	no	10	22	yes	7.9	recursion cap
S3 runaway	sidecar	no	10	22	yes	8.1	recursion cap
S4 storm	off	yes	49	0	no	30.8	—
S4 storm	wipe	no	0	2	yes	1.5	burst entropy
S4 storm	sidecar	no	0	2	yes	1.9	burst entropy
Stale probe	wipe	—	0	1	no	—	allow after wipe
Stale probe	sidecar	—	0	1	yes	—	recursion cap
Fresh S1	wipe	yes	1	3	no	—	reports depth 0
Fresh S1	sidecar	yes	1	3	no	—	reports depth 0

E Synthetic mesh: an invariant, not a field result

Death is load ≥ 1. The simulated gate refuses any request with load + request > 0.95. Under that pair of rules a node cannot die: 100% survival is an invariant of the construction. Without pre-accumulation refusal, the same seeds collapse (mean 1.876/500 alive at $T = 100$ over 1,000 runs). The 100% cells are not a field result.

Table 8: Mean survival (%) at $T = 100$. Breaker-only matches the full gate.

Malicious %	Full	No v floor	No breaker	No burst tax	Breaker only	v only / base
5	100	100	4.8	100	100	0.3 / 0.3
20	100	100	15.7	100	100	0.2 / 0.2

Table 9: Mean survival (%) versus malicious fraction (300 runs).

Mode	1%	5%	10%	15%	20%	30%	40%	50%
Baseline	2.3	0.3	0.3	0.3	0.2	0.2	0.2	0.2
Full gate	100	100	100	100	100	100	100	100

Table 10: Mean survival (%) when malicious per-request load is reduced (5% malicious, 300 runs).

Malicious load	0.80	0.45	0.20	0.12
Baseline	0.3	0.6	3.1	7.8
Full gate	100	100	100	100